

SYSTEM ARCHITECTURE DOCUMENT

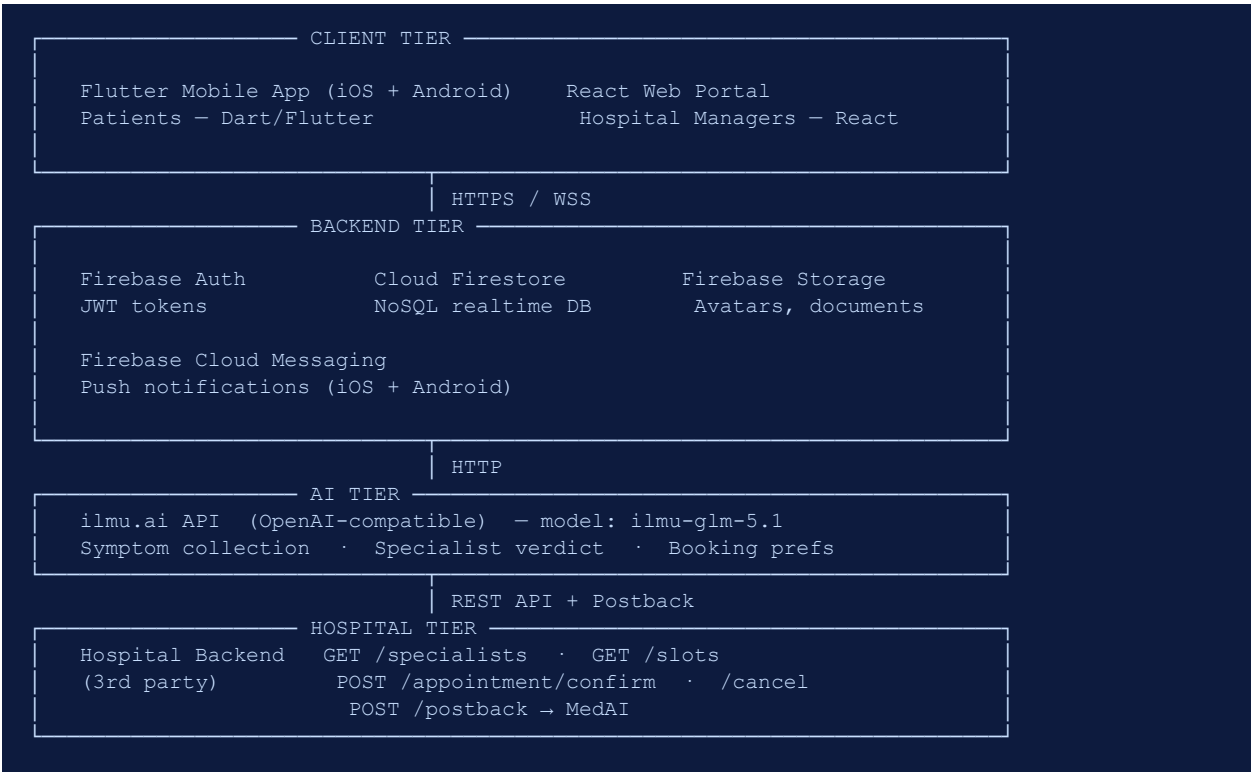
# System Architecture Document

A comprehensive technical specification of the MedAI platform — covering infrastructure, data flows, AI integration, security, and scalability design.

---

**Document:** SAD v1.0    **Backend:** Firebase (Auth, Firestore, Storage, FCM)  
**AI:** ilmu-glm-5.1 via ilmu.ai API    **Mobile:** Flutter (Dart) — iOS & Android

# Architecture Overview



## Technology Stack

| Layer              | Technology                 | Version / Notes                      |
|--------------------|----------------------------|--------------------------------------|
| Mobile UI          | Flutter + Dart             | SDK ≥ 3.0, Provider state management |
| Web Portal         | React                      | Frontend-only, no backend required   |
| Authentication     | Firebase Auth              | Email/password, JWT tokens           |
| Database           | Cloud Firestore            | v25.1.4, real-time streams           |
| File Storage       | Firebase Storage           | Avatars, medical documents           |
| Push Notifications | Firebase Cloud Messaging   | iOS (APNs) + Android (FCM)           |
| AI / NLP           | ilmu-glm-5.1 via ilmu.ai   | OpenAI-compatible API, JSON mode     |
| HTTP Client        | Dart http package          | v1.2.2                               |
| Image/File Picker  | image_picker + file_picker | Camera, gallery, PDF                 |
| Secure Storage     | flutter_secure_storage     | API keys, sensitive tokens           |

## Mobile App — Component Architecture

```
lib/
├─ main.dart          Firebase.initializeApp · MultiProvider · RootScreen routing
├─ firebase_options.dart Platform-specific Firebase credentials
├─ core/
│   └─ constants.dart  API keys, base URLs, model name
├─ models/
│   └─ user_model.dart  Pure Dart data classes with Firestore serialization
├─ appointment_model.dart AppointmentStatus + StatusSource enums
├─ hospital_model.dart
├─ specialist_model.dart
├─ slot_model.dart      formattedDate/Time via intl
├─ chat_message_model.dart ChatMessageType: text|image|document|booking|slot|confirm
├─ services/            Stateless singletons, no Flutter dependencies
│   └─ auth_service.dart  Firebase Auth wrapper
│   └─ firestore_service.dart All Firestore CRUD + streams
│   └─ storage_service.dart  Firebase Storage (avatars, documents)
│   └─ ai_service.dart      ilmu.ai HTTP calls + response parsing
│   └─ mock_data_service.dart Seeds 6 hospitals + specialists + slots
├─ providers/           ChangeNotifier — reactive state layer
│   └─ auth_provider.dart   Auth state stream + onboarding flag
│   └─ chat_provider.dart    8-state AI chat state machine
│   └─ appointments_provider.dart Firestore real-time stream
├─ screens/
│   └─ onboarding/
│       └─ auth/            sign_in · register · forgot_password
│       └─ home/
│       └─ ai_chat/
│       └─ appointments/
│       └─ profile/
├─ widgets/             hover_button · hover_scale · platform_utils
```

## AI Chat State Machine



## AI Integration Design

### Symptom Collection

- OpenAI-compatible chat completion API
- System prompt instructs collection of 6 fields: mainComplaint, duration, severity (1–10), associatedSymptoms, medicalHistory, medications
- Completion detected via `<SYMPTOMS_COMPLETE>{json}</SYMPTOMS_COMPLETE>` tag
- Full conversation history passed each call for context

### Verdict Generation

- JSON mode request for structured output
- Returns: specialty, urgency, explanation
- Urgency maps to: ASAP / Within a week / Routine
- Fallback to "General Practice" on API failure

## Security Architecture

### Authentication & Authorization

- Firebase Auth issues JWT tokens; all Firestore reads/writes verify request.auth.uid
- Users can only read/write their own documents
- Hospital data is read-only for authenticated users
- API keys stored in flutter\_secure\_storage, never in plain SharedPreferences
- ilmu.ai API key stored server-side only (in app constants, not exposed in JS)

### Production Firestore Rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{userId} {
      allow read, write: if request.auth.uid == userId;
    }
    match /appointments/{id} {
      allow read, write: if request.auth.uid == resource.data.userId;
    }
    match /hospitals/{id} {
      allow read: if request.auth != null;
    }
    match /aiChats/{id} {
      allow read, write: if request.auth.uid == resource.data.userId;
    }
  }
}
```

### Data Privacy

- Patient passport & phone stored in Firestore (encrypted at rest by Firebase)
- Medical documents uploaded to Firebase Storage with per-user path isolation: documents/{userId}/...
- AI chat history stored per-user; no cross-user data access
- Only free slots fetched from hospitals — no patient/occupancy data transferred

### Transport Security

- All Firebase SDK traffic over TLS 1.3
- Hospital API calls use Bearer token in Authorization header
- Android: network\_security\_config enforces HTTPS
- iOS: App Transport Security (ATS) enforced

### Hospital API Security

- Each hospital receives a unique Bearer token from MedAI
- Token validated on every incoming request by hospital backend
- Postback URL protected with shared secret header

## Scalability Design

| Concern            | Design Decision                                                | Scales To                         |
|--------------------|----------------------------------------------------------------|-----------------------------------|
| Patient database   | Cloud Firestore — serverless, auto-sharded                     | Millions of users                 |
| Slot availability  | slotCache collection with 1-hour TTL; batched hospital fetches | 1000s of hospitals                |
| AI calls           | Stateless HTTP; conversation history managed client-side       | Concurrent users × API rate limit |
| File storage       | Firebase Storage CDN — per-user paths                          | Petabyte-scale                    |
| Push notifications | FCM topics / device tokens via Firestore                       | Billions of devices               |
| Real-time updates  | Firestore onSnapshot() — WebSocket streams                     | Low-latency at scale              |

## Deployment & CI/CD

| Mobile Releases                                                                                    | Firebase Rules                                                                                               | Environment Config                                                                                                    |
|----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Flutter build pipelines via GitHub Actions. Android APK → Play Store; iOS IPA → App Store Connect. | Firestore & Storage rules versioned in firestore.rules. Deployed via firebase deploy --only firestore:rules. | Firebase project-per-environment (dev / staging / prod). firebase_options.dart generated per env via FlutterFire CLI. |